

Chapter 6. 합성곱 신경망 II - Part 2(6.1.5~)

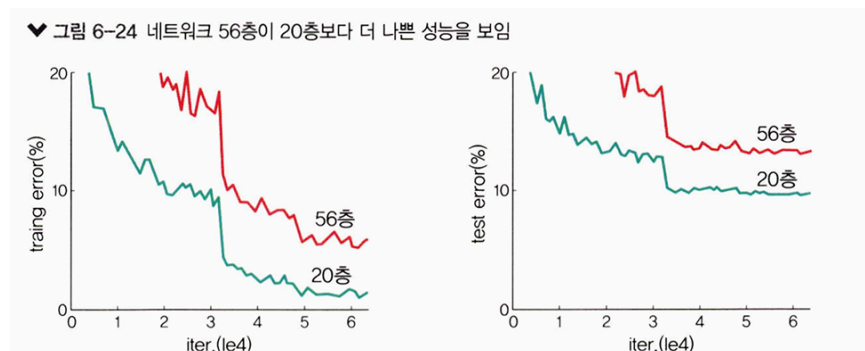
6.1 이미지 분류를 위한 신경망

Chapter 6. 합성곱 신경망 II - Part 1(~6.1.4)

6.1.5 ResNet

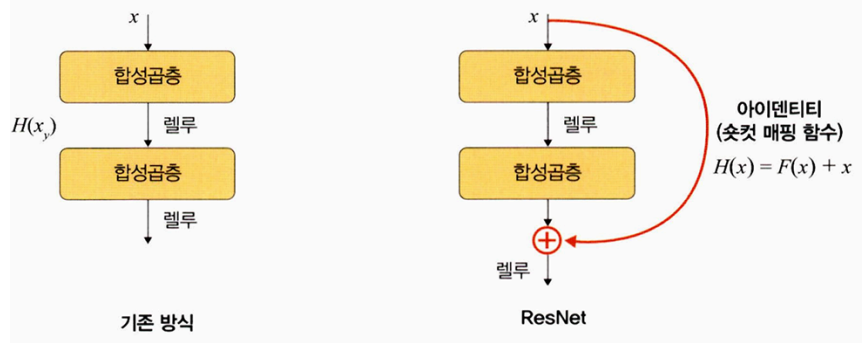
개요

- 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 **residual** 개념 고안
- 신경망이 깊이가 깊어질수록 성능이 좋아지지만 일정 단계에 다다르면 성능이 나빠짐



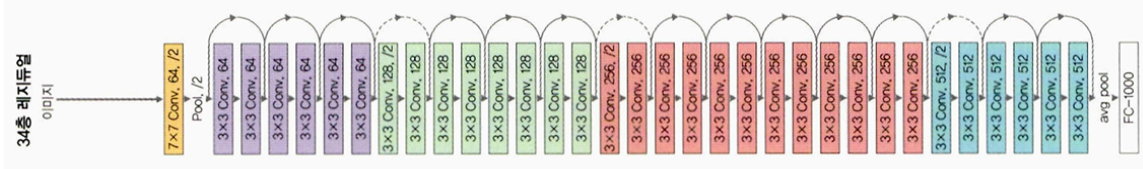
- 네트워크 깊이가 깊다고 해서 무조건 성능이 좋지 않음
 - ⇒ 이 문제를 해결하기 위해 ResNet은 **residual block** 도입
 - residual block: 기울기가 잘 전파될 수 있도록 일종의 shortcut(skip connection)을 만듦
- shortcut을 두어 기울기 소멸 문제 방지

▼ 그림 6-25 ResNet 구조



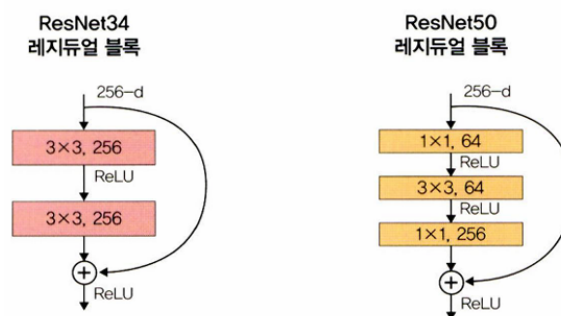
- block: 계층의 묶음(합성곱층의 묶음)

▼ 그림 6-26 ResNet 모델 전체 네트워크



- 묶인 계층들을 하나의 residual block이라 하고, 이를 여러 개 쌓은 것이 ResNet
- 계층의 깊이가 깊어질 수록 파라미터 수가 무제한으로 커지는 문제 → bottleneck block
 - 병목 블록(bottleneck block)

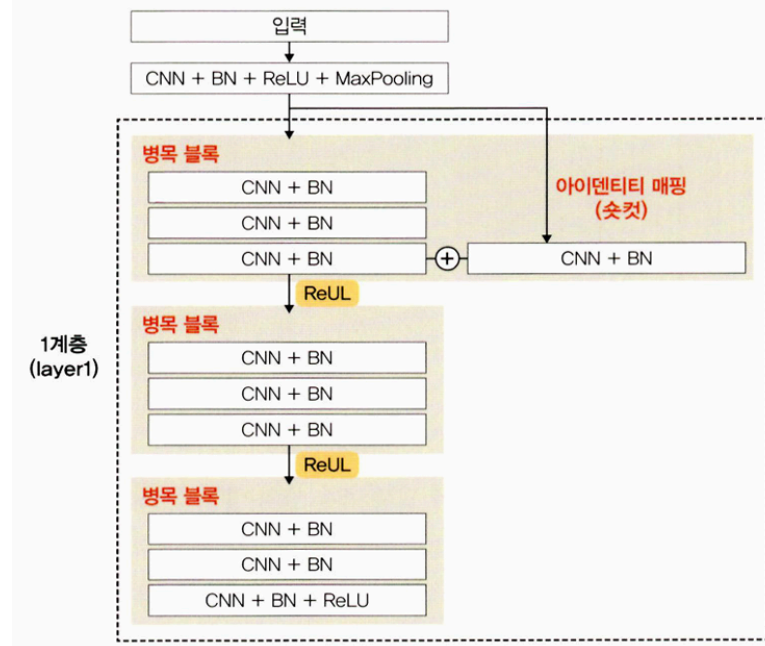
▼ 그림 6-27 기본 블록과 병목 블록



- 병목 블록 사용: 파라미터 수가 감소하는 효과
- 원리: 앞뒤로 1x1 합성곱층 → 채널 수를 조절하며 차원을 조절
- + 기호: 아이덴티티 매핑
- **identity mapping**(shortcut, skip connection)

입력 x가 어떤 함수를 통과하더라도 다시 x라는 형태로 출력되게 함

▼ 그림 6-28 아이덴티티 매핑(숏컷)



```
def forward(self,x):
```

```
    i = x
```

```
    x = self.conv1(x)
```

```
    x = self.bn1(x)
```

```
    x = self.relu(x)
```

```
    x = self.conv2(x)
```

```
    x = self.bn2(x)
```

```
    if self.downsample is not None:
```

```
        i = self.downsample(i) #다운샘플 적용
```

```
    x += i #아이덴티티 매핑 적용
```

```
    x = self.relu(x)
```

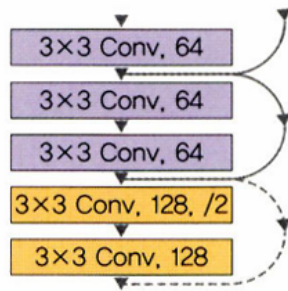
```
    return x
```

if) x : (28, 28, 64)

x를 i 변수에 저장했기 때문에 (28, 28, 64), 그리고 합성곱층을 통과하면서 같은 형태를 더하기 때문에 최종 형태는 (28, 28, 64)

- **downsample**: feature map 크기를 줄이기 위한 것, 풀링과 같은 역할

♥ 그림 6-29 ResNet 네트워크의 일부



- 노란색 영역 시작 지점에서 채널 수가 128로 늘어났고, /2라는 것으로 보아 첫 번째 블록에서 합성곱의 스트라이드가 2로 늘어나 (14, 14, 128)로 바뀐다는 것을 알 수 있음

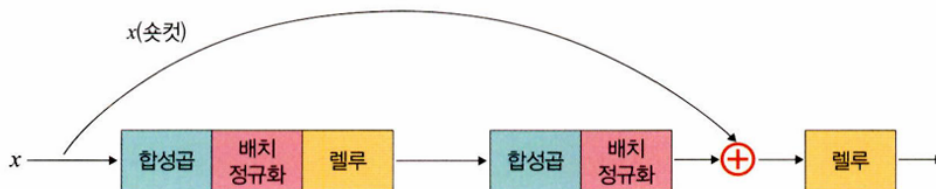
→ 보라색과 노란색의 형태가 다른데 이들 간의 형태를 맞추지 않으면 아이덴티티 매핑X

→ 아이덴티티에 대한 다운샘플이 필요

- **아이덴티티 블록:** 입력과 출력의 차원이 같은 것

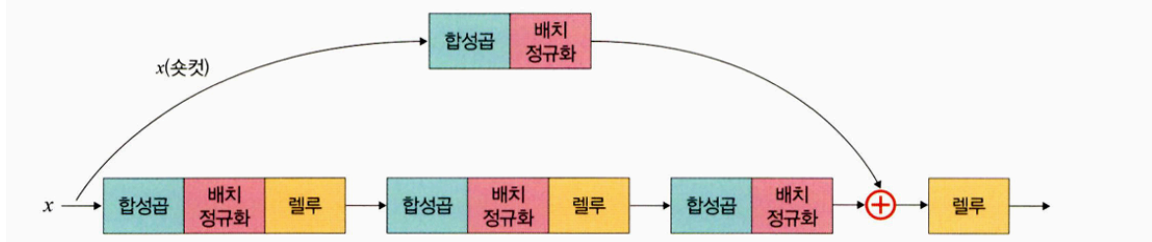
- 입력과 차원의 형태를 맞추어 주기 위해 stride 2를 가진 1x1 합성곱 계층을 하나 연결

♥ 그림 6-31 아이덴티티 블록



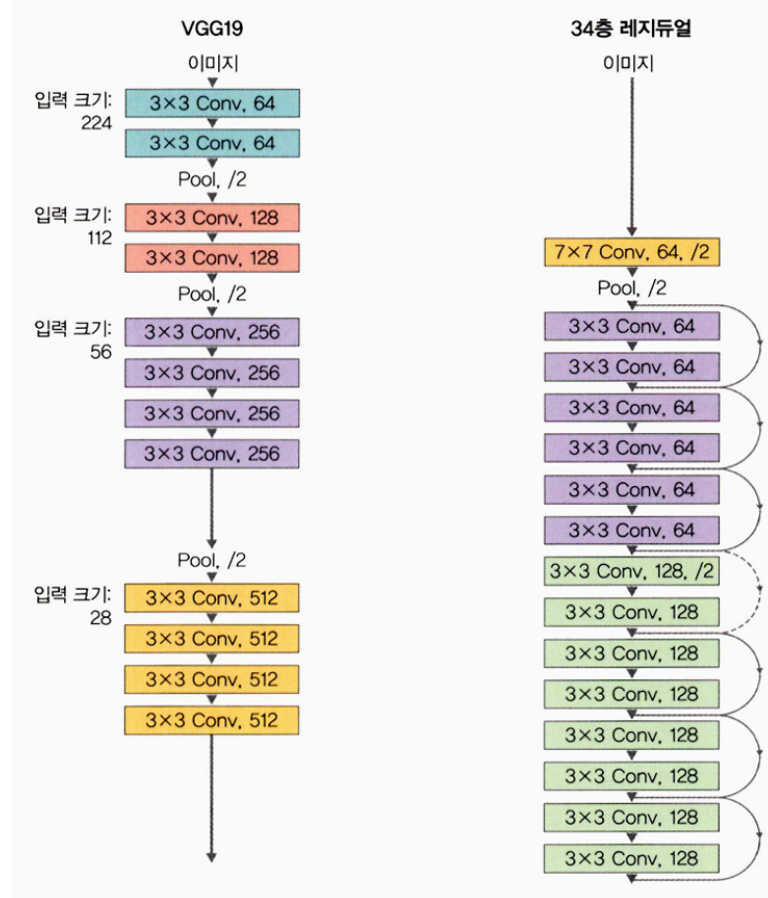
- **projection-shortcut(합성곱 블록):** 입력 및 출력 차원이 동일 X, 입력의 차원을 출력에 맞추어 변경해야 하는 것

♥ 그림 6-30 합성곱 블록



- **(정리)** ResNet은 VGG19 구조가 뼈대, 합성곱층들을 추가해서 깊게 한 후 숏컷들 추가

♥ 그림 6-32 VGG19와 ResNet 비교



☀ ResNet 예제 구현을 해보자!

#코드 6-65. 필요한 라이브러리 호출

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.models as models

import matplotlib.pyplot as plt
import numpy as np

import copy
```

```

from collections import namedtuple #
import os
import random
import time

import cv2
from torch.utils.data import Dataset, DataLoader
from PIL import Image

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

- ① namedtuple: 튜플의 성질을 갖지만 인덱스 뿐만 아니라 키 값으로도 데이터 접근

(데이터 전처리 과정은 앞선 모델과 매우 유사하여 과정 설명 생략)

#코드 6-73. 기본 블록 정의

- 기본 블록은 ResNet18, ResNet34에서 사용되며 합성곱(3x3) 두개로 구성
- downsample 적용

```

if downsample:
    conv = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                     stride=stride, bias=False)
    bn = nn.BatchNorm2d(out_channels)
    downsample = nn.Sequential(conv, bn)
else:
    downsample = None
self.downsample = downsample

```

- 네트워크를 통과한 후, 입력 데이터 크기와 출력 데이터 크기 다를 경우 사용
- 합성곱에 스트라이드 적용
- identity mapping(skip connection) 적용

```

def forward(self, x):
    i = x
    x = self.conv1(x)

```

```

x = self.bn1(x)
x = self.relu(x)
x = self.conv2(x)
x = self.bn2(x)

if self.downsample is not None:
    i = self.downsample(i)

x += i #아이덴티티 매핑 적용
x = self.relu(x)
return x

```

- 특정 층에 존재하는 출력 결과를 다음 합성곱층을 통과한 출력 결과에 더함
- x에 conv1, bn1, relu,... 값이 더해지다가 초기의 x가 다시 더해지는 것

#코드 6-74. 병목 블록 정의

- ResNet50, ResNet101, ResNet152에 사용
- 1×1 합성곱층, 3×3 합성곱층, 1×1 합성곱층으로 구성
- ResNet에서 병목 블록을 정의하는 하이퍼파라미터: **expansion**



기본 블록 → 병목 블록 변경 이유

기본 블록 구조	병목 블록 구조
3×3 합성곱층 두 개	1×1 합성곱층, 3×3 합성곱층, 1×1 합성곱층

- 계층을 더 깊게 쌓으면서 계산에 대한 비용 감소
- 계층 ↑ → 활성화 함수 ↑ → 비선형성 처리 ↑
⇒ 다양한 입력 데이터에 대한 처리 가능

#코드 6-75. ResNet 모델 네트워크

- 각 residual branch 마지막의 BN을 0으로 초기화 → 다음 branch에서 0으로 시작

```

if zero_init_residual:
    for m in self.modules():
        if isinstance(m, Bottleneck):
            nn.init.constant_(m.bn3.weight, 0)
        elif isinstance(m, BasicBlock):
            nn.init.constant_(m.bn2.weight, 0)

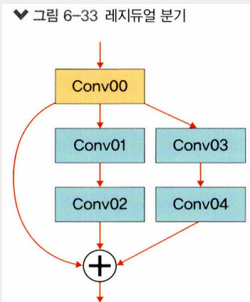
```

- BN을 0으로 초기화할 경우 모델 성능이 0.2~0.3% 향상 → ResNet에서 많이 사용



Residual Branch

프로그램의 조건에 따라 A, B, C 등으로 분기하는 것



#코드 6-76. ResNetConfig 정의

- 네임드튜플 데이터 형식으로 ['block','n_blocks','channels'] 저장

#코드 6-77. 기본 블록을 사용하여 ResNetConfig 정의

- 기본 블록을 사용하는 ResNet18, ResNet34d의 Config 정의
- 블록은 BasicBlock 사용, 블록과 채널의 크기는 각각 지정

#코드 6-78. 병목 블록을 사용하여 ResNetConfig 정의

- 병목 블록을 사용하는 ResNet50, ResNet101, ResNet152의 Config 정의
- 블록은 Bottleneck 사용, 블록과 채널의 크기는 각각 지정

(#코드 6-79. 사전 훈련된 ResNet 모델 사용, 코드 6-80. 사전 훈련된 ResNet네트워크 확인

- ResNet은 사전 훈련된 모델이므로 간단하게 사용할 수도 있음
- 사전 훈련된 모델 사용 방법)

#코드 6-81. ResNet50 Config을 사용한 ResNet 모델 사용

- Config부분에 resnet50_config 지정
- OUTPUT_DIM에 테스트 데이터셋 클래스 입력
- 모델 확인 : 사전 훈련된 모델 네트워크와 비교했을 때 차이X → 간단하게 사전 훈련된 모델을 사용해도 좋음!

#코드 6-82. 옵티마이저와 손실 함수 정의

- lr = 1e-7로 정의 : 1×10^{-7} 승

#코드 6-83. 모델 학습 정확도 측정 함수 정의

- torch.topk: torch.argmax와 같은 효과

```
_, top_pred = y_pred.topk(k, 1)
```

- 주어진 텐서에서 가장 큰 값의 인덱스 반환
= 네트워크의 출력에서 가장 확률이 높은 값의 인덱스 반환

- t(): 차원 0과 1을 전치

```
top_pred = top_pred.t()
```

- 텐서 비교 함수

```
correct = top_pred.eq(y.view(1, -1).expand_as(top_pred))
```

- torch.eq: 텐서가 서로 같은지 비교
- torch.ne : 텐서가 서로 다른지 비교
- torch.ge: 텐서가 크거나 같은지 비교

- 형식: `torch.eq(비교 대상 텐서, 비교할 텐서)` → True/False로 반환

(모델 학습과정, 결과 예측 구현은 앞선 모델과 매우 유사하여 과정 설명 생략)

#코드 6-89. 모델 예측에 대한 결과 출력

- 정확도가 높지 않음 → 성능 향상을 위해 데이터 증가 필요

객체를 분류하는 방법

가장 중요한 것은 내가 가진 데이터에 가장 적합한 모델 선택하기!

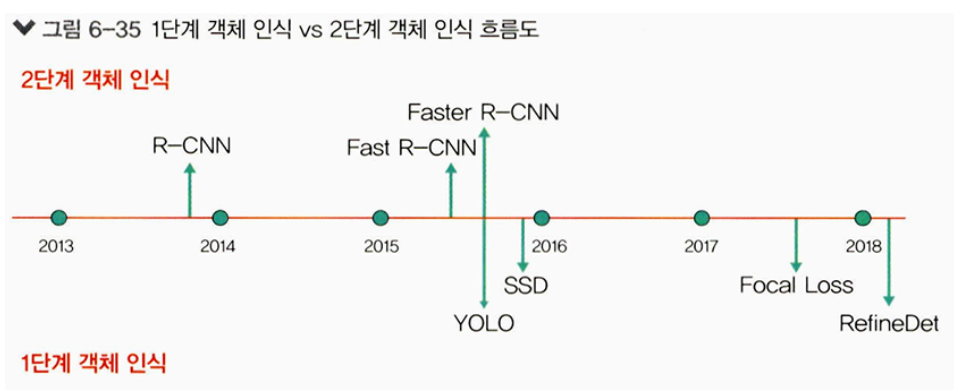
- 구현된 모델을 재사용할 수 있는 것이 많음
- 전이 학습을 사용하여 약간의 튜닝 진행 후 학습

6.2 객체 인식을 위한 신경망

객체 인식 (= 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출)

이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술

- 딥러닝을 이용한 객체 인식 알고리즘



- 1단계 객체 인식: 분류와 위치 검출 동시에 행하는 방법
 - 2단계에 비해 빠르지만 정확도가 낮음
 - YOLO, SSD 계열
- 2단계 객체 인식: 분류와 위치 검출을 순차적으로 행하는 방법

- 1단계에 비해 느리지만 정확도가 높음
- R-CNN

6.2.1 R-CNN

객체 인식 알고리즘은,

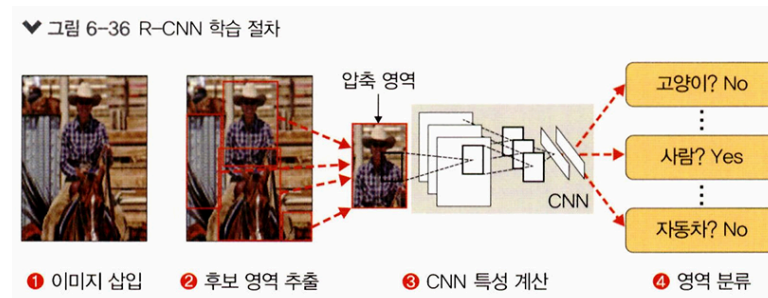
과거) sliding window 방식: 일정한 크기를 갖는 윈도우로 이미지의 모든 영역 탐색하며 객체 검출

현재) selective search 알고리즘을 적용한 후보 영역(region proposal) 사용

R-CNN(Region-based CNN)

이미지 분류를 수행하는 CNN과 이미지에서 객체가 있을 만한 영역을 제안해 주는 후보 영역 알고리즘을 결합한 알고리즘 ◦ 지

- R-CNN의 수행 과정



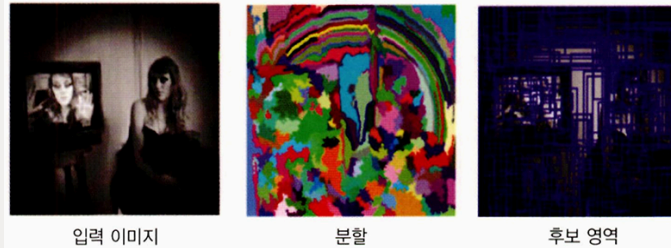
1. 이미지 입력
2. 2000개의 바운딩 박스를 선택적 탐색 알고리즘으로 추출한 후 잘라 내고, CNN 모델에 넣기 위해 같은 크기(227x227)로 통일
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델 적용
4. 각각 분류를 진행하여 결과 도출



선택적 탐색

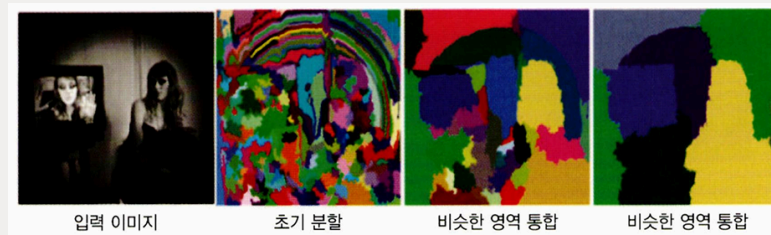
- 객체 인식이나 검출을 위한 가능한 후보 영역을 알아내는 방법
- 분할 방식을 이용하여 시드를 선정하고, 그 시드에 대한 완전 탐색을 적용
- 선택적 탐색의 세 단계 과정

1. 초기 영역 생성(sub-segmentation)



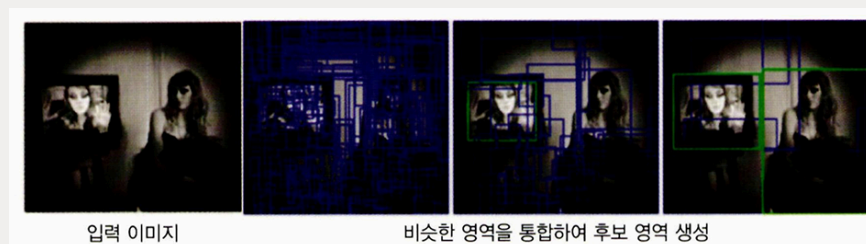
- 각각의 객체가 영역 한 개에 할당될 수 있도록 많은 초기 영역 생성
- 입력된 이미지를 영역 다수 개로 분할하는 과정

2. 작은 영역의 통합



- 1단계에서 영역 여러 개로 나눈 것들을 비슷한 영역으로 통합
- 탐욕 알고리즘을 사용하여 비슷한 영역이 하나로 통합될 때까지 반복

3. 후보 영역 생성



- 2단계에서 통합된 이미지들을 기반으로 후보 영역 추출

- 용어 의미

exhaustive search	후보가 될 만한 대상의 크기 및 비율이 모두 다른 상황을 고려하여 후보 영역을 찾는 기법
segmentation	영상 데이터의 특성에 따라 분할하여 후보 영역을 선정하는 기법
후보 영역(바운딩 박스)	3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
seed	영상에서는 특정 기준범의 픽셀에서 점점 의미가 같은 영상 범위까지 픽셀을 확장해 나가면서 분할하는데, 이때 특정 기준점이 되는 픽셀

- R-CNN의 단점

- 세 단계의 복잡한 학습과정, 긴 학습 시간+대용량 저장 공간, 객체 검출 속도 문제
- 이를 해결하기 위한 Fast R-CNN 생성

6.2.2 공간 피라미드 풀링

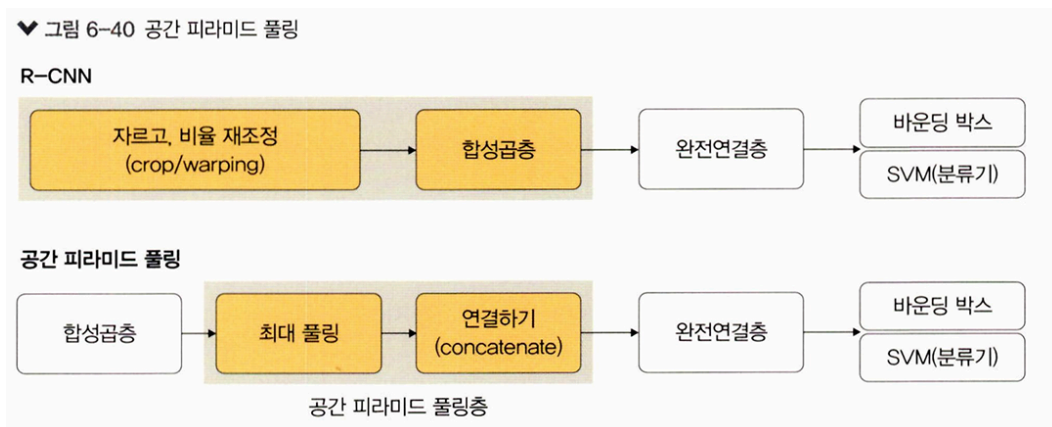
도입 배경

기존 CNN 구조는 완전연결층을 위해 입력 이미지 고정을 위해 자르거나 비율 조정

→ 물체의 일부분이 잘리거나 본래의 생김새와 달라지는 문제점

공간 피라미드 풀링(spatial pyramid pooling)

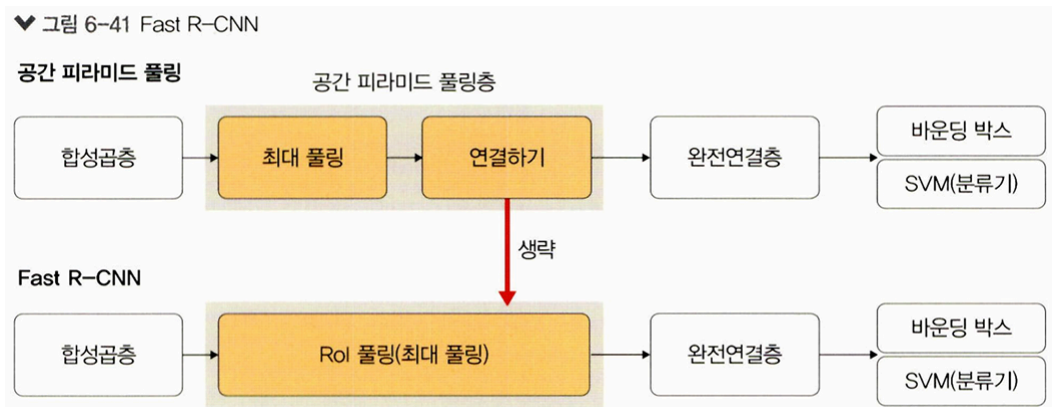
입력 이미지 크기 상관X 합성곱층 통과, 완전연결층에 입력되기 전에 특성 맵들을 동일한 크기로 조절해 주는 풀링층을 적용하는 기법



- 원본 이미지 특징 훼손X

6.2.3 Fast R-CNN

- R-CNN의 속도 문제 개선을 위해 RoI 풀링 도입



- 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지
- 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기 조정



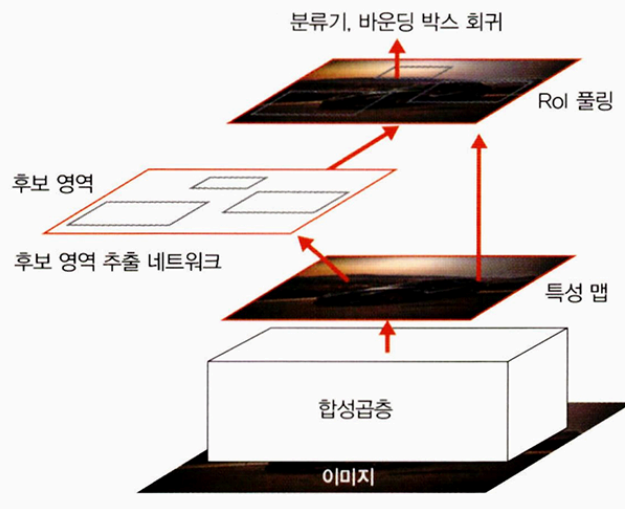
RoI 풀링

크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 결괏값 크기를 동일하게 맞추는 방법

6.2.4 Faster R-CNN

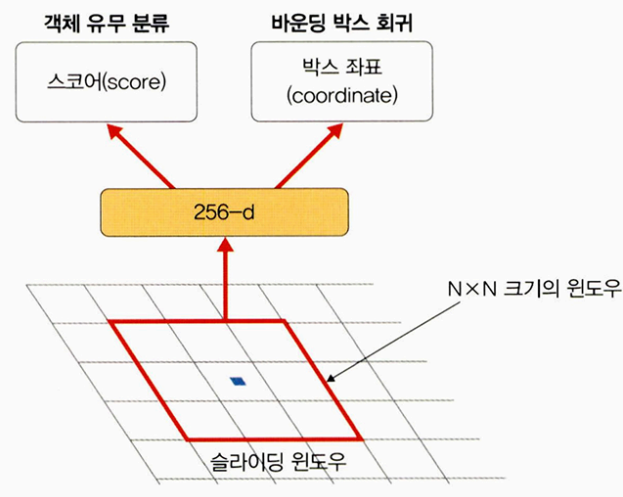
- 더욱 빠른 객체 인식을 수행하기 위한 네트워크
- 기존 Fast R-CNN의 후보 영역 생성을 CNN 내부 네트워크에서 진행하도록 설계
⇒ Faster R-CNN은 기존 R-CNN에 후보 영역 추출 네트워크 추가
- 외부의 느린 선택적 탐색(CPU) 대신 내부의 빠른 RPN(GPU로 계산)dmf tkdyd
- RPN은 마지막 합성곱층 다음에 위치, RoI풀링과 분류기, 바운딩 박스 회귀가 위치

♥ 그림 6-43 Faster R-CNN

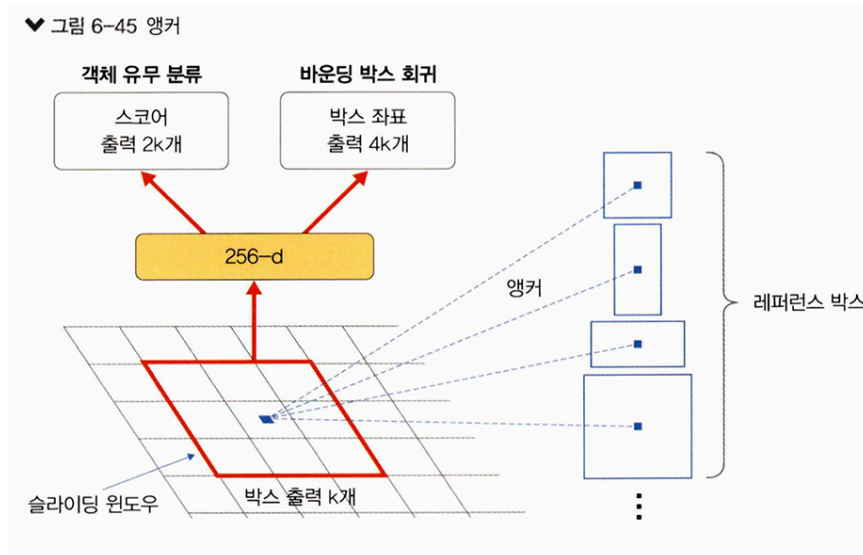


- 후보 영역 추출 네트워크

♥ 그림 6-44 후보 영역 추출 네트워크



- 특성 맵 $N \times N$ 크기의 작은 윈도우 영역을 입력받고, 해당 영역에 객체의 존재 유무 판단을 위해 이진 분류를 수행하는 작은 네트워크 생성
- 바운딩 박스 회귀 추가 → 위치 보정을 위해
- 슬라이딩 윈도우 방식으로 작은 윈도우 영역을 이용해 객체 탐색
- 단점: 다양한 크기와 비율의 이미지 수용이 어려움
→ 이를 보완하기 위해 앵커 방식 사용
- 앵커: 여러 크기와 비율의 레퍼런스 박스 k 개 미리 정의, 슬라이딩 윈도우 위치마다 박스 k 개 출력하도록 설계



- 후보 영역 추출 네트워크의 출력 값은 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 2k 개의 분류에 대한 출력과 x, y, w, h 위치 보정 값을 위한 4k개의 회귀 출력을 가짐

6.3. 이미지 분할을 위한 신경망

이미지 분할(image segmentation)

이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출

6.3.1 완전 합성곱 네트워크

이미지 분류에서 좋은 성능의 CNN 기반 모델을 변형시켜 이미지 분할에 적합하도록 만든 네트워크

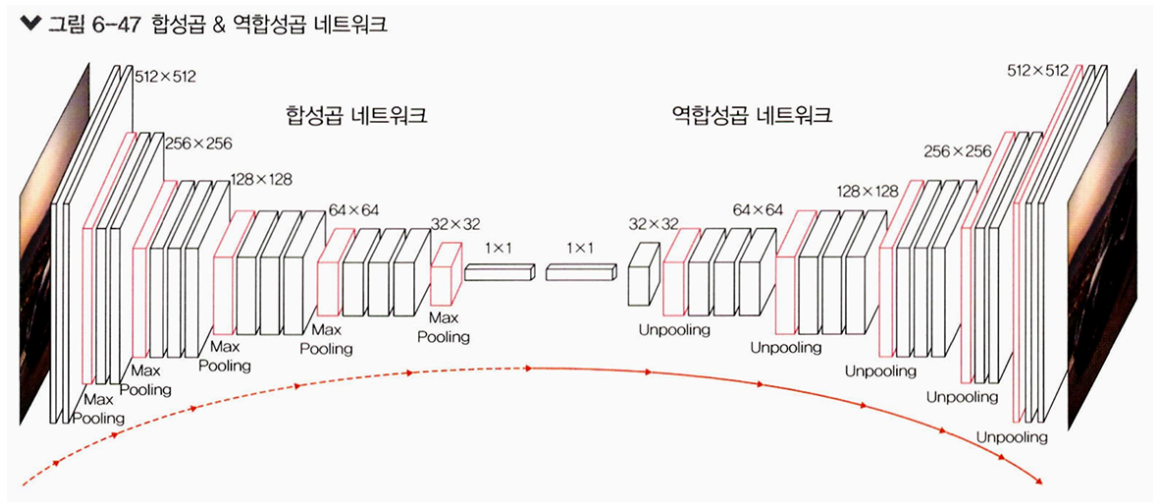
- 장점
 - 예)완전연결층 세 개를 1x1 합성곱으로 변환 → 위치 정보O, 히트맵으로 객체 위치 확인
 - 합성곱층으로 사용 → 입력 이미지에 대한 크기 제약이 사라짐
- 단점
 - 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아짐
 - 낮아진 해상도를 복원하기 위해 업 샘플링 방식을 사용하기 때문에 이미지의 세부 정보들을 잃어버리는 문제 발생

⇒ 이를 해결하기 위한 합성곱 & 역합성곱 네트워크

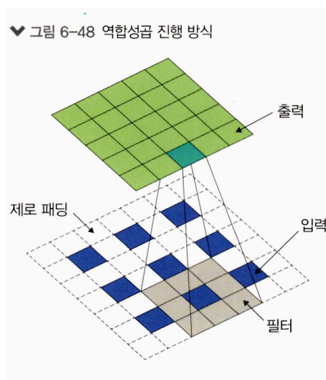
6.3.2 합성곱 & 역합성곱 네트워크

역합성곱

- 역합성곱: 특성 맵 크기 증가(↔ 합성곱층: 특성 맵 크기 축소)



- CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들 때 사용
- 시멘틱 분할 등에 활용
- upsampling이라고도 함
- 역합성곱 동작 방식

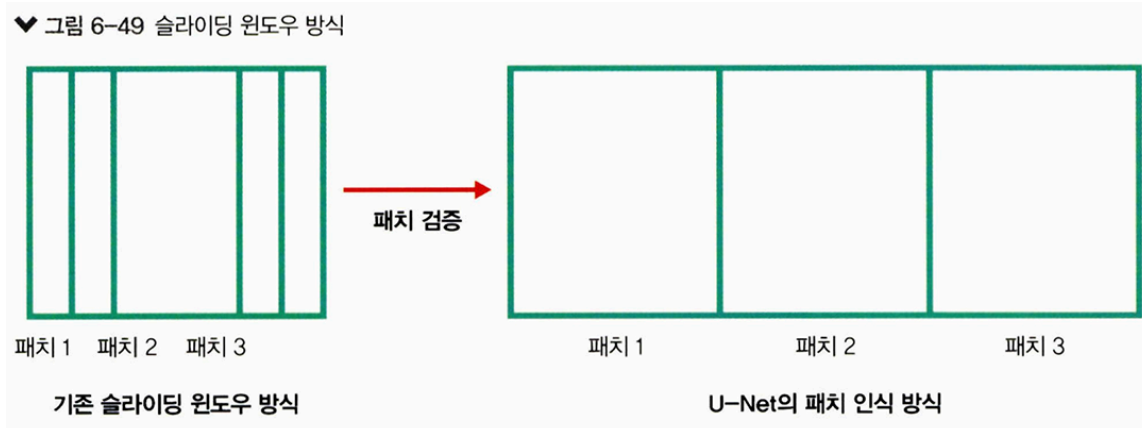


1. 각각의 픽셀 주위에 제로 패딩 추가
2. 패딩된 것에 합성곱 연산 수행
 - 파란색 픽셀 주위로 흰색 제로 패딩 수행
 - 회색 필터로 합성곱 연산을 렉

6.3.3 U-Net

U-Net의 특징

- 속도가 빠르다: 이미 검증이 끝난 패치는 건너뛰다



- 트레이드오프에 빠지지 않는다: 컨텍스트 인식과 지역화 트레이드오프 문제 개선

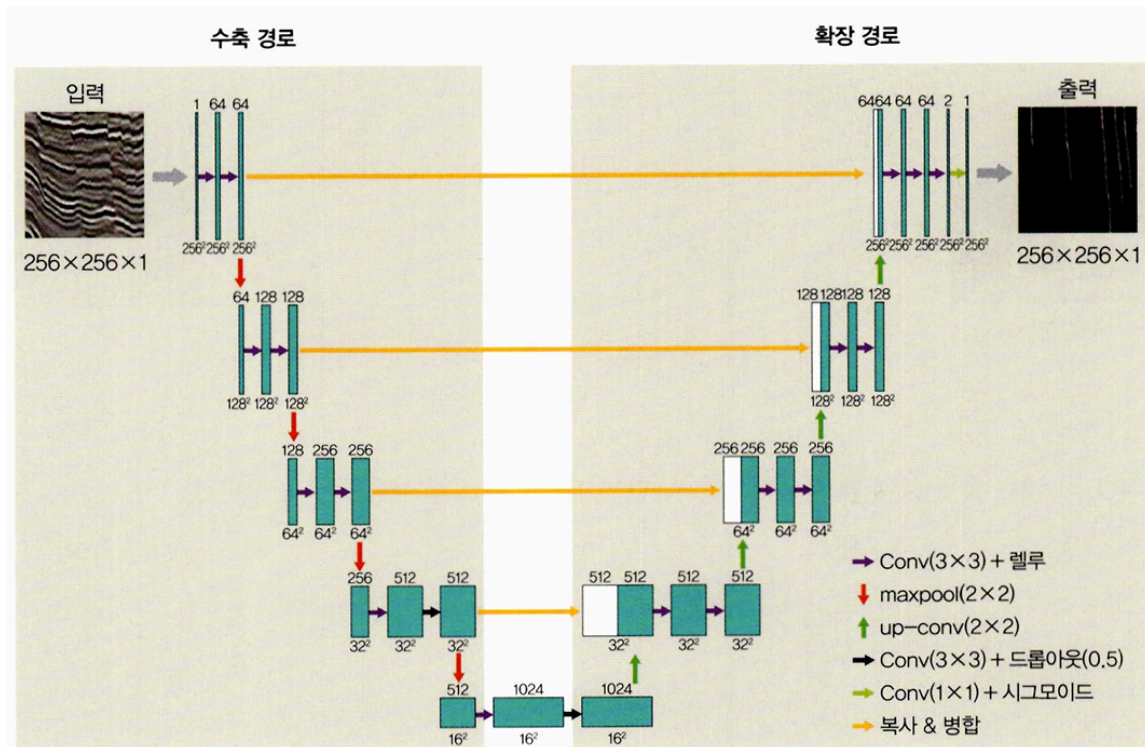


지역화

- 이미지 안에 객체 위치 정보를 출력해 주는 것으로, 주로 바운딩 박스 사용
- 바운딩 박스의 네 꼭지점 픽셀 좌표가 출력되는 것이 아닌 왼쪽 위, 오른쪽 아래 좌표를 출력

U-Net의 구조

- FCN 기반으로 구축, 수축 경로와 확장 경로로 구성
 - 수축 경로: 컨텍스트 포착
 - 확장 경로: 특성 맵을 업 샘플링, 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합하여 정확한 지역화를 수행
- 3×3 합성곱이 주를 이룸
- 각 합성곱 블록은 3x3 두 개로 구성, 그 사이에 드롭아웃



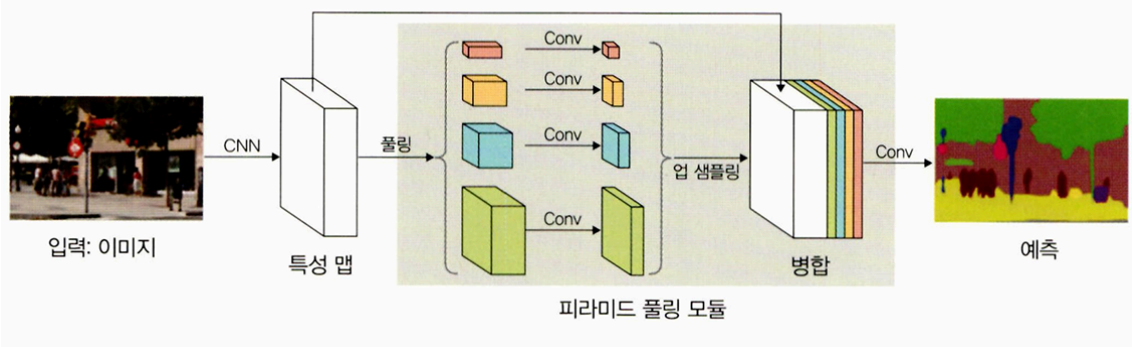
- 수축 경로의 블록
 - 3×3 합성곱 두 개로 구성된 것이 네 개 있는 형태
 - 각 블록은 최대 풀링을 이용하여 크기를 줄이며, 다음 블록으로
- 확장 경로의 블록
 - up-conv
 - 수축 과정에서 줄어든 크기를 다시 키워 가며 합성곱 블록 이용

⇒ 크기가 다양한 이미지 객체 분할을 위해 다운 샘플링과 업 샘플링 반복

6.3.4 PSPNet

- 완전연결층 한계를 극복하기 위해 풀링 모듈을 추가
- 훈련 과정

♥ 그림 6-52 PSPNet



1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링 진행
2. 1×1 합성곱을 사용하여 채널 수 조정, 풀링층 개수가 N 일 때 출력 채널 수는 입력채널 수/ N
3. 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링(*양선형 보간법 사용)
4. 원래의 특성 맵과 새로운 특성맵 병합



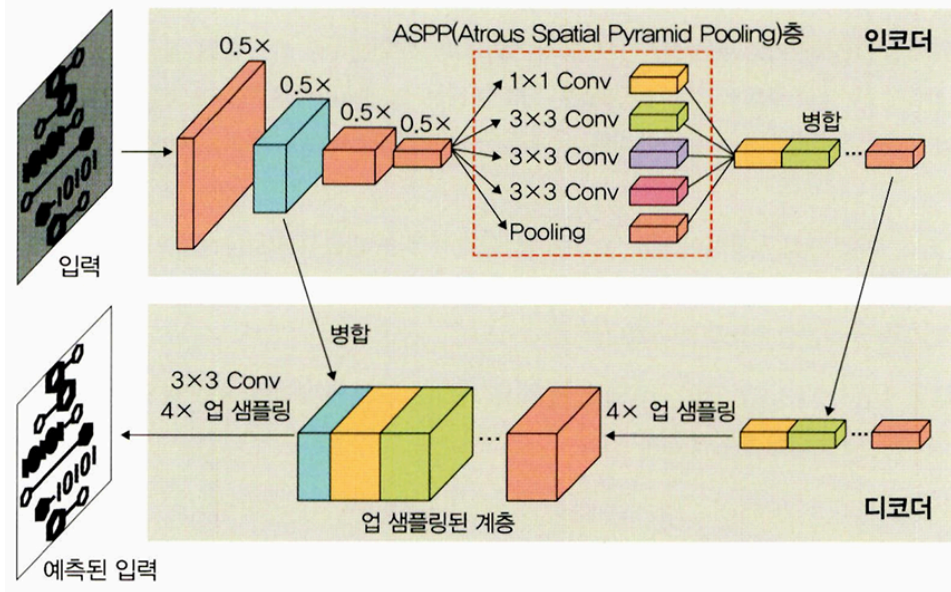
양선형 보간법

- 보간법: 빈 화소에 값을 할당하여 좋은 품질의 영상을 만드는 방법
- 선형 보간법: 원시 영상의 화소 값 두 개를 사용하여 원하는 좌표에서 새로운 화소 값을 계산하는 방법
- 양선형 보간법: 화소당 선형 보간을 세 번 수행하며, 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치 곱한 값을 합해 얻음

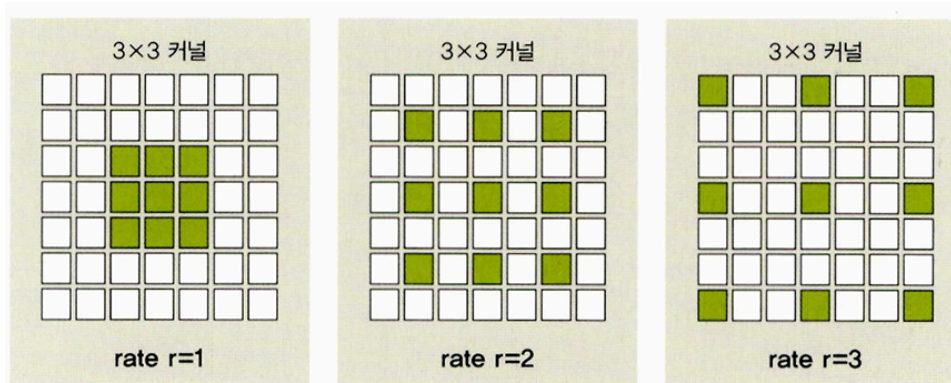
6.3.5 DeepLabv3/DeepLabv3+

- 완전연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용하는 네트워크
- 인코더 디코더 구조
- 인코더에서 추출된 특성 맵의 해상도를 Atrous 합성곱을 도입하여 제어 가능

▼ 그림 6-54 DeepLab의 인코더-디코더 구조

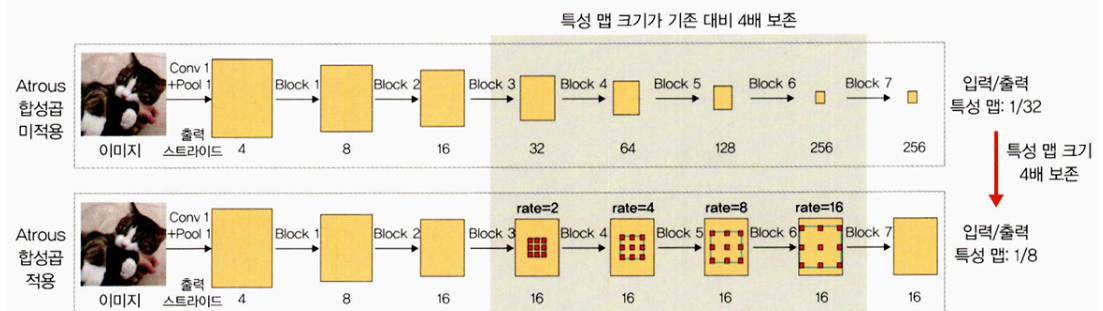


- Atrous 합성곱: 필터 내부에 빈 공간을 둔 채로 동작



- 파라미터 rate: 얼마나 많은 공간을 가질지 결정, r이 클수록 빈 공간이 많아짐
- 파라미터 수를 늘리지 않아도 수용 영역을 크게 키울 수 있어서 많이 사용

▼ 그림 6-56 Atrous 합성곱 효과





수용 영역

외부 자극이 전체에 영향을 주는 것이 아니라 특정 영역에만 영향을 준다는 의미

- 특정 범위를 한정해서 처리하여 효과적으로 훈련을 수행